

GenAI Legacy Code Refactoring – Business Plan

by Dinis Cruz, Chat GPT Deep Research and Claude Opus 4 Research

Meta Information

This is a proposed business model for the idea of creating a company focused on refactoring legacy code. This document is released under the CC Zero license in the hope that it helps entrepreneurs who want to implement this idea. This represents how I (Dinis Cruz) would build such a company, leveraging the technologies, methodologies, and philosophies I've developed through projects like The Cyber Boardroom, MGraph-DB, and OSBot.

Executive Summary

GenAI Legacy Code Refactoring is a SaaS company dedicated to modernizing and refactoring legacy software codebases using generative AI (GenAI), human expertise, and automated workflows. Our platform focuses on safely transforming old, untested, and convoluted code into better-tested, well-documented, and more maintainable code – without changing its external behavior.

We achieve this through a three-phase process:

- **Phase 1:** Adding comprehensive test coverage and documentation
- **Phase 2:** Ensuring continuous integration (CI) pipelines run smoothly
- **Phase 3:** Performing AI-assisted code refactoring under the safety net of rigorous tests

This approach targets the universal pain point in software development: legacy code that is costly to maintain and risky to change. By leveraging GenAI with human-in-the-loop reviews in a serverless architecture, we deliver this service at a transparent, usage-based price point that makes it accessible for organizations worldwide. Our business model is pure SaaS with virtually zero fixed infrastructure costs, enabling global scalability and flexible pricing. We foresee tremendous demand across industries – initially focusing on codebases under 1 million lines – and minimal direct competition in offering human-augmented AI refactoring as a service. GenAI Legacy Code Refactoring is poised to unlock significant value by reducing technical debt, extending the lifespan of critical applications, and freeing engineering teams to focus on innovation instead of fighting with legacy code.

Market Opportunity and Problem Statement

Virtually every medium-to-large organization struggles with legacy code – systems that are essential but difficult to maintain and enhance. Legacy code is often defined not by age but by lack of tests and clarity: "Legacy code is any code base that has no unit tests"¹. Such codebases accumulate technical debt, become brittle, and stifle innovation. Businesses currently spend an enormous share of IT budgets maintaining these systems instead of building new value. In fact, organizations allocate an estimated 60–80% of IT budgets just to keep legacy systems running². This status quo is financially draining and unsustainable.

The pain of legacy maintenance is felt in cost, risk, and time-to-market. Without proper tests or documentation, even small changes carry high risk of breaking functionality. Teams waste countless hours deciphering tangled code logic. Technical debt costs can average \$361k per 100,000 lines of code – meaning a million-line codebase may quietly "owe" millions in refactoring work³. Traditional modernization approaches (like full rewrites or "lift-and-shift" cloud migrations) are costly and disruptive, often requiring long downtime or complete system replacements⁴. Many organizations avoid these options, choosing to "cope" with legacy systems at the expense of agility.

Manual refactoring and cleanup of legacy code is extremely slow and failure-prone. Large refactoring projects often run over budget and past deadlines, or even fail outright. Studies show that projects relying on manual code rewriting are six times more likely to fail compared to those using automated conversion⁵. One real-world example: refactoring a legacy VB6 application to modern standards took 3.5 years and cost over \$750,000 – an effort few companies can afford to repeat⁶. For many organizations, their legacy applications have reached a state where it's nearly impossible to make any changes – creating massive risks when security vulnerabilities are discovered or business requirements change. Clearly, a new approach is needed to modernize legacy software faster, safer, and more affordably.

These challenges create a huge market opportunity. Virtually every industry – finance, healthcare, government, technology, etc. – is sitting on aging codebases that need renewal. We estimate tens of thousands of codebases globally under 1 million lines of code as our initial serviceable market, spanning mid-size applications common in enterprises. These systems are big enough to be mission-critical yet small enough to refactor in a reasonable time with automation and human expertise. By targeting codebases of this scale across all industries (and later scaling up to larger ones), GenAI Legacy Code Refactoring can address a ubiquitous need. The timing is perfect: recent advances in GenAI now make it feasible to automate code understanding and transformation tasks that were impractical to do at scale before. Industry leaders are already hinting at this direction – e.g. NTT Data notes that GenAI can automate core modernization tasks like code refactoring, performance tuning, and security fixes, allowing incremental improvements that preserve business logic⁴. The demand for solutions is clear, and our company is uniquely positioned to deliver an answer.

Solution Overview: Automated Refactoring with Human-in-the-Loop

GenAI Legacy Code Refactoring offers an AI-driven pipeline augmented by human expertise to transform a legacy application without altering its functionality. It's crucial to understand that this is not a "fully automated" service – by design, we incorporate experienced architects, developers, testers, QA and security professionals throughout the process. Customers can provide these experts from their FTE base, or we can facilitate access to amazing freelancers through platforms like Upwork, partner with consulting firms, or collaborate with development-focused companies that have this talent, adding a transparent 15-20% markup for coordination services.

Our service operates in three coordinated phases, each adding value and de-risking the next step. Throughout these phases, our prime directive is: no tests shall fail and there shall be zero change in external behavior of the software. All work is performed on a forked GitHub repository (public or

private), regardless of where the original source code resides. Git provides version control while GitHub manages code, issues, CI pipeline executions, secrets, and more. This creates clear handover points through pull requests to upstream repositories when code reaches appropriate maturity.

Phase 1: Testing, Analysis & Documentation

In Phase 1, we add a comprehensive safety net to the legacy codebase and gain a deep understanding of its structure:

Automated Test Coverage with Bug-First Approach: We generate extensive unit, integration, and end-to-end tests for the existing code. Critically, all known bugs and vulnerabilities must have passing tests that capture their current behavior. This "bug-first" testing approach (developed by Dinis Cruz) ensures tests are meaningful – real bugs encountered in the system are first captured by a test that passes with the bug present and would fail once the bug is fixed⁸⁹. Any broken or non-functioning tests are fixed or updated to reflect the current expected behavior. The goal is to achieve near-100% code coverage, so that every part of the application is exercised by tests. High coverage gives developers confidence to evolve the code – a comprehensive test suite immediately shows the side-effects of any change, enabling confident code evolution⁷. The better our test framework, the more boldly we can refactor without fear, since the tests will catch any behavioral changes.

CI Pipeline MVP: Phase 1 also establishes a basic CI pipeline focused primarily on test execution. Depending on what already exists, this might be a minimal viable CI system that runs the tests automatically. Note that in some cases, it may not be possible to run full integration tests due to lack of existing CI-friendly build/deploy systems or absence of test/QA endpoints. This limitation is addressed more comprehensively in Phase 2.

Automated Code Analysis & Documentation: In tandem with testing, we leverage GenAI to thoroughly document the codebase. Our platform reads through every source file, configuration, and build script using large language models, and produces detailed technical documentation for each component. We essentially create a knowledge base of what the code does, in plain language and in rich formats (diagrams, maps, etc.). LLMs are capable of accurately interpreting code – for example, Dinis Cruz has demonstrated an AI (Claude) reading a class's source code and generating correct and expressive documentation of its behavior¹⁰. We harness this ability at scale. For every function, module, and configuration, the AI generates a clear description of its purpose and logic. The entire codebase is indexed and cross-referenced for easy querying.

Architecture Visualization with Human Calibration: We go beyond text documentation by producing in-depth analyses and visualizations of the system architecture. Using semantic analysis (powered by an open-source memory-first graph database and LLMs), we build a knowledge graph of the code¹¹. This graph represents relationships between components and can be used to answer high-level questions about the software structure. Critically, these graphs, ontologies, and taxonomies are reviewed, calibrated, pruned, and refined by human experts. The service doesn't need to get everything right initially – human-in-the-loop reviews and adjustments are essential and would be impossible without the AI-generated foundation. We generate diagrams such as dependency graphs

and UML charts to illustrate how different parts of the application interact¹². The platform can produce natural-language summaries of architectural layers¹³.

Value of Phase 1: The outputs of Phase 1 dramatically reduce uncertainty. The test suite acts as a guardrail – any unintended change in behavior will be caught immediately. The documentation and maps act as a compass – guiding our engineers (and the client's team) through the complexity of the code. Many clients may find value in Phase 1 alone: even before any refactoring, they get a fully tested codebase with comprehensive bug and vulnerability tests, plus up-to-date documentation (a huge leap in maintainability)¹⁴.

Phase 2: Build Integration & Pipeline Modernization

Phase 2 ensures that the application's build and deployment process is modern, reliable, and automated. We integrate the testing and analysis from Phase 1 into the development workflow:

CI/CD Pipeline Enhancement: We enhance or establish a comprehensive Continuous Integration pipeline so that the application can be built from source and all tests can run automatically on each change. This phase focuses on creating the actual build code and deployable artifacts. If the project already has CI, we update it to ensure smooth execution of the new test suite. If there was no CI, we provide a ready-made serverless CI setup. The goal is a one-button (or one-commit) build and test execution, giving rapid feedback on code changes.

Environment and Dependency Updates: Often legacy projects suffer from outdated build scripts, missing dependencies, or environment drift. We address these issues so the software can be built in a clean, reproducible environment (e.g. using containerization or standardized cloud build services). This may involve updating config files, build tools, or minor code tweaks purely for compatibility – all validated by the tests to ensure no behavior change.

Security Assessment Integration: Phase 2 includes an initial security review of the codebase using existing SAST tools and GenAI code review workflows. Each vulnerability discovered is documented and added as a "passing test" that captures the vulnerable behavior. It's important to note that fixing these vulnerabilities is out of scope for this project – our goal is to discover, document, and create tests for them. We analyze whether vulnerabilities in third-party dependencies actually affect the current codebase (they might be in unused code/functionality), providing valuable risk assessment data.

End-to-End Testing on New Builds: With the pipeline in place, we execute full end-to-end tests on the application in a production-like environment. This serves as a final check that Phase 1 and 2 outputs are correct – the application still passes all behavioral tests in an environment similar to deployment.

Value of Phase 2: This phase ensures that we have a solid operational foundation. The client gains a modern CI/CD setup with integrated security assessment, improving their DevOps maturity. Phase 2 clears the path for large-scale code changes – we can refactor with confidence knowing that a robust pipeline will catch regressions immediately, and we have visibility into existing security issues.

Phase 3: AI-Driven Code Refactoring & Modernization

In Phase 3, we perform the core mission of the company – refactoring the legacy code to improve its maintainability using GenAI assistance, human expertise, and without breaking functionality:

Safe, Behavior-Preserving Refactoring with Human Oversight: Guided by the documentation, tests, and existing architecture principles provided by human experts, our tooling systematically refactors the codebase. This process is highly agentic-AI friendly, allowing multiple models and reasoning loops between each pull request. Depending on budget, various LLMs can be employed to leverage the best capabilities of each. Every change is made through focused, reviewable pull requests – not too large, allowing for human-in-the-loop or LLM-in-the-loop review workflows.

Refactoring activities include restructuring code into clearer modules, renaming variables and functions for clarity, simplifying complex functions, removing dead or duplicate code, and applying modern design patterns where appropriate. A critical rule during this process is that no test can fail – we refuse to change the externally observable behavior. Each change is validated against the test suite. This approach follows the classic refactoring principle of small, incremental changes – but at machine scale with human validation.

Pragmatic Improvements with Realistic Expectations: It's important to note that there will always be limitations on how much refactoring can achieve, depending on the application's code, allocated time/tokens/budget, and human-in-the-loop bandwidth. The goal is not "clean code with zero technical debt" (a subjective and often unattainable standard) but rather to leave the application in a measurably better state than originally found. Success is measured through pragmatic OKRs rather than absolute claims.

GenAI Support for Refactoring: GenAI plays a pivotal role in Phase 3, making the refactoring process smarter and more efficient. Our AI models can review the entire codebase and identify improvements while preserving the intent of the code. The AI can adapt test code automatically when implementation details change, ensuring the test's intent remains the same¹⁵. This capability to co-evolve tests with code is crucial for avoiding "brittle tests" that would otherwise hinder refactoring. We leverage AI to remember and preserve bug fixes and edge-case handling – known bug patterns in the code are not lost during cleanup¹⁶.

Continuous Test Improvement: As we refactor, we continuously improve the test suite as well. New code paths introduced by refactoring get new tests. Because the code becomes cleaner and more modular, we can write more fine-grained unit tests that were previously impossible. Documentation is updated to reflect structural changes through re-running our Phase 1 documentation generation on the refactored code.

Value of Phase 3: The refactored system provides significant long-term value. Developers can onboard to the project more easily using the documentation and make changes with confidence using the extensive tests. The business can introduce new features or address security vulnerabilities much faster on this improved foundation. We've extended the viable life of the application by making it

maintainable again – as one industry analysis put it, GenAI allows businesses to "modernize incrementally while extending the lifespan of their critical legacy systems"⁴.

Pragmatic Success Metrics

Rather than making absolute claims about outcomes, we measure success through pragmatic OKRs (Objectives and Key Results) that can be tailored to each project:

Code Quality Metrics:

- Test coverage increase (e.g., from 0% to 80%+)
- Reduction in cyclomatic complexity by X%
- Decrease in code duplication by Y%
- Number of documented functions/modules (from 0 to 100%)
- Reduction in average function length
- Increase in modularity score

Operational Metrics:

- Time to run full test suite
- Build time improvement
- Number of security vulnerabilities identified and tested
- CI pipeline reliability (% successful builds)
- Time to onboard new developers (measured through surveys)

Business Impact Metrics:

- Reduction in time to implement new features
- Decrease in production incidents after changes
- Developer satisfaction scores
- Estimated technical debt reduction (in monetary terms)
- Number of previously "untouchable" modules now safely modifiable

For some applications, even minor improvements represent major success stories – especially for systems where changes were previously nearly impossible due to fear of breaking unknown dependencies.

Technology and Operations

GenAI Legacy Code Refactoring's platform is built on a serverless, scalable architecture that keeps costs low and throughput high. We have developed a unique automation pipeline that orchestrates code analysis, test generation, and code transformation using a combination of cloud functions, AI models, and graph-based data stores. Key technology highlights include:

Git-Centric Workflow: All work is performed on forked GitHub repositories, providing clear version control, issue tracking, and handover points through pull requests. This approach works regardless of the original source location and provides transparency throughout the process.

Generative AI Workflows: At the core, we use Large Language Models (like GPT-4, Claude, and others) in a controlled, deterministic way to perform code reasoning tasks. These AI agents are orchestrated through prompts and tooling to read code, generate tests, suggest refactors, and produce documentation. We emphasize determinism and provenance – every AI-generated output is validated and traceable. By combining AI suggestions with strict test verification and human review, we maintain high quality in all outputs.

Semantic Code Knowledge Graph: We utilize a semantic knowledge graph to represent the code structure and metadata. All analysis from Phase 1 (ASTs, function docs, dependencies, call graphs) is stored in this graph. We employ MGraph-DB, a memory-first graph database designed by Dinis Cruz for AI and serverless use cases¹⁷¹⁸. The knowledge graph allows us to ask complex questions about the code and drives both the documentation generator and refactoring decisions. For public code, we publish and maintain relevant ontologies and taxonomies; for private code, clients own these graphs entirely.

Serverless Infrastructure: Our entire platform runs on a serverless architecture in the cloud (using AWS Lambda, Fargate, etc., and managed services). This means we have no always-on servers. Compute resources scale up when a customer's codebase is being processed and scale down to zero when idle¹⁹. The result is an extraordinarily lean operation: when there are no active refactoring jobs, our baseline compute cost is near-zero.

Open-Source Tooling and Extensibility: We stand on the shoulders of proven open-source tools and our platform itself is open source. We use industry-standard parsing libraries, testing frameworks, and CI/CD integrations. Some of our utilities (e.g. the Type_Safe modeling library from the OSBot project²⁰) are already open-sourced. Our architecture is multi-cloud and portable²¹.

Security and Privacy: Customer code is sensitive IP, so we isolate each project's data completely. For privacy-sensitive repositories, we recommend running our open-source platform in the customer's environment – either through cloud marketplace images (like AWS AMI) or in a dedicated cloud account provided at subscription cost. AI models can be configured to run in privacy-preserving modes. Our approach often improves security awareness by identifying and testing vulnerabilities (though fixing them remains out of scope unless specifically requested).

Business Model

GenAI Legacy Code Refactoring operates under a pure SaaS model with transparent usage-based pricing and optional partner-led services:

Token-Based Pricing with Transparent Markup: We charge based on the actual tokens used during processing, adding a consistent 20% markup to all model costs (including free models like Groq or SambaNova). This ensures we always generate revenue while giving clients full control over costs by

choosing which models to use. Clients are never exposed to unexpected overruns – they set the budget and model preferences. This "dynamic cost pass-through" approach means that changes in underlying AI costs are transparently reflected²³.

Subscription Options: We're exploring base subscription tiers (from \$0 to \$X per month) that provide platform access, with all processing costs additional. This model ensures every client is profitable – the only question is margin size. Initial pricing discovery will happen through early client engagements.

Human-in-the-Loop Services: When clients need additional expertise, we can facilitate access to freelance architects, developers, and security professionals through platforms like Upwork, partner with consulting firms, or collaborate with development-focused companies that have this talent, adding a transparent 15-20% coordination markup. This allows us to remain a product company while enabling necessary human expertise.

Low Cost Structure and Scalability: Thanks to our serverless architecture, the company incurs minimal fixed costs²⁴. We have eliminated most fixed operating expenses – no data centers or idle server fleets. Our costs scale only with paying customers (compute, AI API calls, storage), covered by usage fees. This gives us massive operating leverage and competitive pricing ability.

Global Reach, Multi-Region SaaS: Being cloud-based, our service is available worldwide from day one. We'll deploy in multiple regions to comply with data residency requirements. The platform connects to code repositories and initiates processing without on-premise installation in the standard model.

Partner-Enabled Extensions: We establish a partner network of consulting firms and independent experts who can provide value-added services on top of our platform. These partners handle bespoke needs while we remain focused on the core product. Partners might use our tool as a backend to deliver custom reports or handle post-refactoring manual tweaks if required by the client.

Go-to-Market Strategy

To launch and grow GenAI Legacy Code Refactoring, we will pursue a targeted go-to-market plan:

Initial Target Segment – Mid-size Codebases in Tech-Savvy Firms: Our sweet spot is codebases under ~1 million lines of code. We target companies maintaining such systems with acute modernization needs. Ideal early adopters include technology companies with legacy modules, fintech and insurance companies, and software consultancies that might use our tool for their clients.

Open Source Demonstrations: We'll refactor several open-source projects to demonstrate real value, ideally obtaining testimonials from project maintainers. These serve as proof points and allow potential customers to see actual results before committing.

Free Assessment Offer: We offer free initial codebase assessments using free LLM APIs (Groq, SambaNova) to lower barriers for new customers. This might include basic analysis of a code subset, showing potential improvements and estimated effort saved. Our cost for such assessments is minimal, serving as both marketing and technical pre-sales.

Thought Leadership and Content Marketing: We'll establish expertise through webinars, whitepapers, and conference talks, leveraging existing research on semantic code analysis, GenAI in AppSec, and serverless architectures. Case studies will show before-and-after results with permission.

Risk Mitigation Through Incremental Adoption: Clients mitigate risk by starting small with pilot applications. All changes require client approval and ownership – we provide tools and expertise, but clients retain full responsibility for their code. For sensitive repositories, we offer deployment in customer environments.

Competitive Positioning: We differentiate from traditional consulting (faster, cheaper, more complete) and partial automation tools (end-to-end process with human expertise). We embrace open-source tools as enablers rather than competitors.

Partnerships: We'll seek partnerships with DevOps platforms (GitHub, GitLab), cloud providers (AWS, Azure, GCP), and potentially create regional/specialized subsidiaries (e.g., "COBOL in Brazil" or "COBOL for Financial Services") to address specific market segments with domain expertise.

Financial Projections and Funding

Our financial strategy emphasizes profitability from day one through usage-based pricing and near-zero fixed costs. Key principles:

Revenue Model: Token usage markup (20%) plus potential base subscriptions. Every project is profitable by design. Human services coordination adds additional markup opportunity.

Cost Structure: Variable costs only (AI tokens, compute, storage) covered by usage fees. No fixed infrastructure or large team overhead.

Scalability: Serverless architecture means we can serve global demand without capital investment. Break-even is easily attainable due to low fixed costs.

Future Opportunities: Potential for specialized regional/language companies, continuous monitoring subscriptions, marketplace models, and knowledge base products.

With thousands of potential projects globally, revenue potential reaches hundreds of millions annually when scaled. The model is designed for bootstrapping or efficient use of seed funding focused on product development rather than operational burn.

Team Requirements and Company Vision

While this company has not been created yet, successful implementation requires a founding team with similar background and expertise to what has been demonstrated through projects like The Cyber Boardroom, OSBot, and MGraph-DB. The ideal team should:

Technical Leadership: Deep experience in software architecture, testing, CI/CD, and security. Understanding of both legacy and modern development practices. Proven ability to leverage GenAI for code analysis and transformation.

Open Source Philosophy: Commitment to transparency and community contribution. Many of the tools and methodologies described here are open source and should remain so.

Serverless Expertise: Understanding of cloud-native, serverless architectures to maintain the zero-fixed-cost model that makes this business sustainable.

Domain Knowledge: Either directly or through advisors, access to expertise in specific legacy technologies (COBOL, mainframes, etc.) and industry verticals.

Implementers should leverage as many of the ideas, technologies, and approaches documented in Dinis Cruz's work as possible, as these form the philosophical and technical foundation for this business model.

Conclusion

GenAI Legacy Code Refactoring represents a timely solution to a massive and growing problem in the software industry. By combining AI capabilities with human expertise in a transparent, usage-based model, we make legacy modernization accessible to organizations of all sizes.

Our three-phase approach – adding tests and documentation, modernizing build pipelines with security assessment, and performing incremental refactoring – addresses technical debt systematically while minimizing risk. The emphasis on human-in-the-loop review ensures quality and builds trust, while our serverless architecture keeps costs low and scalable.

Success is measured not through absolute transformation claims but through pragmatic improvements that deliver real business value. Whether enabling previously impossible changes to critical systems or simply making maintenance less painful, every improvement matters.

The opportunity is enormous and ripe for execution. Organizations worldwide struggle with legacy code that constrains their ability to innovate and respond to change. By focusing on this problem with the latest in GenAI, cloud technology, and human expertise, GenAI Legacy Code Refactoring can become an indispensable partner for any enterprise looking to revitalize its software assets.

This business model is offered freely under CC Zero license to inspire entrepreneurs who recognize this opportunity. The combination of technological readiness, market need, and sustainable business model makes this the right time to transform how the world deals with legacy code.

References

1. Feathers, Michael (via LinkedIn post comment): "Legacy code is any code base that has no unit test for them."
2. GAO Report via DataCenterDynamics – 60–80% of IT budget is spent on maintaining existing on-site hardware and legacy apps
3. NTT Data – Average technical debt cost is \$361k per 100k lines; traditional "rip and replace" modernization is high-risk

4. NTT Data – Generative AI is automating tasks like code refactoring in legacy app modernization, allowing incremental updates that preserve business logic
5. TurinTech (Artemis) – Manual legacy code rewrites are 6x more likely to fail vs automated conversion
6. TurinTech (Artemis) – one VB6 app refactor took 3.5 years, \$750k
7. Dinis Cruz – 100% test coverage with GenAI support enables confident code evolution; tests immediately reveal side-effects of changes
8. Dinis Cruz – "Bug-first" testing approach to make tests especially meaningful
9. Beyond 100% Code Coverage: How GenAI and "Bug-First" Testing Transform Software Quality
10. Dinis Cruz – LLM (Claude) was able to read source code and produce accurate documentation of its behavior
11. Dinis Cruz – Semantic Knowledge Graphs for LLM-Driven Source Code Analysis
12. Spyrosoft Case Study – AI-powered refactoring suggestions improved code performance, readability, maintainability; also generated UML diagrams
13. Dinis Cruz – The AI-generated knowledge graph can be used to answer questions and produce high-level architecture descriptions
14. Spyrosoft Case Study – Used GenAI to automatically analyze and document legacy code
15. Dinis Cruz – When refactoring, GenAI helps adapt tests to new implementations, preserving bug patterns and keeping test intent clear
16. Dinis Cruz – Comprehensive tests lead to cleaner refactoring and more confident deployments
17. Dinis Cruz (MGraph-DB) – Memory-first graph DB can be used in a serverless function
18. Dinis Cruz (MGraph-DB) – Built to be lightweight and serverless-friendly
19. Dinis Cruz – Serverless financial model: eliminating fixed costs while maximizing scalability
20. Dinis Cruz – Type_Safe modeling library from the OSBot project
21. Cyber Boardroom – Multi-cloud and runs-everywhere design
22. Dinis Cruz – Background as CISO and GenAI in AppSec expertise
23. Dinis Cruz – Dynamic cost pass-through model
24. Dinis Cruz – Serverless startup philosophy avoiding fixed expenses
25. TurinTech – Artemis AI tool for code refactoring
26. Spyrosoft – GenAI for code documentation and optimization
27. Cyber Boardroom investment strategy
28. Cyber Boardroom Business Plan
29. Dinis Cruz – GenAI Startup Journey writings
30. Wardley Mapping for AI strategy